Trabajo Práctico - Unidad 4

Forma Normal de Boyce-Codd y desnormalización controlada en Food Store

Base de Datos II - Estudiante: Mateo Algañaraz - Repositorio: MateoAlganaraz/BaseDeDatos2, carpeta **tp6/**

Entorno: PostgreSQL 16.15, base de laboratorio **food_store_tp6_lab** creada desde **schema.sql**. Scripts: **tp_fnbc_control_lote.sql** (Parte 1) y **tp_desnormalizacion_top_categorias.sql** (Parte 2). Todas las cifras provienen de las salidas guardadas en **tp6/evidencia/**; los recuadros grises son capturas textuales de psql.

Parte 1 - FNBC sobre control_lote_almacen

Esquema R(LoteID, DepositoID, ResponsableControlID), PK (lote_id, deposito_id). Las tablas maestras lote, deposito y usuario no existen en el esquema del repositorio; se crean mínimas en un esquema aislado **mayorista** para que las FOREIGN KEY de la consigna sean válidas.

a) Dependencias funcionales

DF1: {LoteID, DepositoID} → {ResponsableControlID}. **DF2:** {ResponsableControlID} → {DepositoID}. LoteID solo no determina nada (un lote se controla desde varios depósitos) y DepositoID tampoco.

b) Clausuras y claves candidatas

Conjunto X	X^+	¿Superclave?
{LoteID}	{LoteID}	No
{DepositoID}	{DepositoID}	No
{ResponsableControlID}	{ResponsableControlID, DepositoID}	No
{LoteID, DepositoID}	{LoteID, DepositoID, ResponsableControlID} (DF1)	Sí, mínima → clave candidata
{LoteID, ResponsableControlID}	{LoteID, ResponsableControlID, DepositoID} (DF2)	Sí, mínima → clave candidata
{DepositoID, ResponsableControlID}	{DepositoID, ResponsableControlID}	No (falta LoteID)

LoteID no aparece en ningún lado derecho, así que integra toda clave. **Claves candidatas:** {LoteID, DepositoID} y {LoteID, ResponsableControlID}. **Primos:** los tres atributos. **No primos:** ninguno.

c) Verificación de FNBC

R está en FNBC si toda DF no trivial X → Y tiene X superclave. DF1 cumple. **DF2 viola FNBC**: es no trivial y $\{ResponsableControlID\}^+ = \{ResponsableControlID, DepositoID\} \neq R$, porque no determina LoteID; en la instancia, 801 aparece en dos filas (lotes 501 y 502). La tabla sí está en 3FN (DepositoID es primo), por eso se usa la definición de FNBC.

d) Anomalías (ejecutadas dentro de BEGIN ... ROLLBACK)

Actualización: "801 pertenece al depósito 30" está repetido en las filas de 501 y 502. Si 801 se traslada al depósito 32 y solo se corrige la fila de 501, la tabla lo acepta y registra a 801 en {30, 32}. **Inserción:** la responsable 803, recién incorporada al depósito 31, no puede registrarse sin un lote (lote_id es NOT NULL y parte de la PK); además se acepta (504, 31, 801), dejando a 801 en {30, 31}. **Borrado:** al eliminar la fila del lote 503 se pierde el único registro de que 802 pertenece al depósito 31.

```
== 3.a Anomalía de actualización ==
BEGIN
UPDATE 1
 responsable_control_id | depositos
------------------------+-----------
                    801 | {30,32}
(1 row)

ROLLBACK
== 3.b Anomalía de inserción: responsable sin lote (se espera ERROR controlado) ==
psql:tp6/tp_fnbc_control_lote.sql:148: NOTICE:  Inserción rechazada (null value in column "lote_id" of relation "control_lote_almacen" violates not-
null constraint): no hay forma de registrar "803 pertenece al depósito 31" sin un lote.
DO
== 3.b Anomalía de inserción: 801 aparece en el depósito 31 y la tabla lo acepta ==
BEGIN
INSERT 0 1
 responsable_control_id | depositos
------------------------+-----------
                    801 | {30,31}
(1 row)

ROLLBACK
== 3.c Anomalía de borrado ==
BEGIN
DELETE 1
 filas_que_registran_802_en_deposito_31
----------------------------------------
                                      0
(1 row)

ROLLBACK
```

Captura 1 - Salida de las tres anomalías (evidencia/01_fnbc_control_lote.txt, sección 3).

e) Descomposición sin pérdida

Con la DF violatoria X → Y, X = {ResponsableControlID}: **R1 = X^+ = responsable_deposito(responsable_control_id PK, deposito_id)** y **R2 = R − (X^+ − X) = control_lote_responsable(lote_id, responsable_control_id; PK compuesta)**. FK: R1 → usuario y deposito; R2 → lote y R1. En R1 la única DF es DF2 con determinante clave; en R2 no queda DF no trivial: ambas en FNBC. Vista de compatibilidad: **vw_control_lote_almacen** = control_lote_responsable NATURAL JOIN responsable_deposito. DF1 no se preserva (sus atributos quedan separados); se la protege con disparadores que rechazan un segundo responsable del mismo depósito para un lote.

f) Unión sin pérdida, migración y verificación

R1 ∩ R2 = {ResponsableControlID} y por DF2 {ResponsableControlID} → R1: el atributo común es superclave (clave) de R1. Por el criterio de superclave sobre el atributo común, cada fila de R2 se combina con exactamente una de R1 y R1 ⋈ R2 = R sin tuplas espurias. La migración verifica DF2 sobre los datos, carga R1 y R2 con SELECT DISTINCT en una transacción y compara la tabla original con la vista en ambos sentidos.

```
== 5. Equivalencia original vs. vista (esperado: 0, 0 y conteos iguales) ==
        control        | filas
-----------------------+-------
 original EXCEPT vista |     0
 vista EXCEPT original |     0
 filas en original     |     3
 filas en vista        |     3
(4 rows)
```

Captura 2 - Equivalencia tabla original vs. vista de compatibilidad: 0 diferencias y 3 filas en ambas.

Pruebas posteriores (revertidas): 803 se registra sin lote; el traslado de 801 se hace en una fila y la vista lo refleja en sus dos lotes; borrar el lote 503 conserva (802, 31); la PK rechaza un segundo depósito para 801 y el disparador rechaza un segundo responsable del depósito 30 para el lote 501.

Parte 2 - Top 5 de categorías por monto vendido en el día

Adaptación al esquema real: id → id_producto / id_categoria / id_pedido; subtotal → cantidad × precio_unitario; eliminado = FALSE → deleted_at IS NULL (convención del TPI; se agregó deleted_at a pedido y detalle_pedido); fecha = CURRENT_DATE → rango del día sobre fecha TIMESTAMPTZ (zona America/Argentina/Buenos_Aires). Instancia: 12 categorías, 10.000 productos, 20.000 clientes, 600.000 pedidos (12.000 con baja lógica), 1.500.000 detalles (6.186 con baja lógica), 1.643 pedidos del día.

a) Consulta original: Execution Time 33,129 ms; nodo dominante Parallel Seq Scan on pedido

El Parallel Seq Scan sobre pedido insume 20,062 ms de los 25,439 ms del subárbol de joins de cada proceso (≈ 79 %): lee los 600.000 pedidos para conservar 1.643 (Rows Removed by Filter: 199452 en cada uno de 3 procesos). Luego hace 1.643 búsquedas en pk_detalle_pedido y 4.091 en producto_pkey, Hash Join con categoría y agrega. Mediana de 30 ejecuciones: 28,695 ms.

```
== 5.2(a) EXPLAIN ANALYZE de la consulta original (ANTES) ==
 QUERY PLAN
------------------------------------------------------------
 Limit  (actual time=30.932..33.100 rows=5 loops=1)
   Buffers: shared hit=22645 read=810
   ->  Sort  (actual time=30.931..33.098 rows=5 loops=1)
         Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=22645 read=810
         ->  Finalize GroupAggregate  (actual time=30.554..33.092 rows=12 loops=1)
               Group Key: c.nombre
               Buffers: shared hit=22645 read=810
               ->  Gather Merge  (actual time=30.474..33.073 rows=36 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=22645 read=810
                     ->  Partial GroupAggregate  (actual time=25.812..26.041 rows=12 loops=3)
                           Group Key: c.nombre
                           Buffers: shared hit=22645 read=810
                           ->  Sort  (actual time=25.750..25.799 rows=1364 loops=3)
                                 Sort Key: c.nombre
                                 Sort Method: quicksort  Memory: 216kB
                                 Buffers: shared hit=22645 read=810
                                 Worker 0:  Sort Method: quicksort  Memory: 61kB
                                 Worker 1:  Sort Method: quicksort  Memory: 61kB
                                 ->  Hash Join  (actual time=0.094..25.439 rows=1364 loops=3)
                                       Hash Cond: (pr.id_categoria = c.id_categoria)
                                       Buffers: shared hit=22629 read=810
                                       ->  Nested Loop  (actual time=0.060..25.203 rows=1364 loops=3)
                                             Buffers: shared hit=22596 read=810
                                             ->  Nested Loop  (actual time=0.045..22.543 rows=1364 loops=3)
                                                   Buffers: shared hit=10321 read=810
                                                   ->  Parallel Seq Scan on pedido ped  (actual time=0.030..20.062 rows=548 loops=3)
                                                         Filter: ((deleted_at IS NULL) AND (fecha >= CURRENT_DATE) AND (fecha < (CURRENT_DATE + 1)))
                                                         Rows Removed by Filter: 199452
                                                         Buffers: shared hit=3702 read=810
                                                   ->  Index Scan using pk_detalle_pedido on detalle_pedido dp  (actual time=0.004..0.004 rows=2
loops=1643)
                                                         Index Cond: (id_pedido = ped.id_pedido)
                                                         Filter: (deleted_at IS NULL)
                                                         Rows Removed by Filter: 0
                                                         Buffers: shared hit=6619
                                             ->  Index Scan using producto_pkey on producto pr  (actual time=0.002..0.002 rows=1 loops=4091)
                                                   Index Cond: (id_producto = dp.id_producto)
                                                   Buffers: shared hit=12275
                                       ->  Hash  (actual time=0.013..0.013 rows=12 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                             Buffers: shared hit=3
                                             ->  Seq Scan on categoria c  (actual time=0.004..0.006 rows=12 loops=3)
                                                   Buffers: shared hit=3
 Planning:
   Buffers: shared hit=30
 Planning Time: 0.344 ms
 Execution Time: 33.129 ms
(50 rows)
```

Captura 3 - EXPLAIN (ANALYZE, BUFFERS) ANTES. Se omitieron los valores (cost=...) para que entre en la página; la salida completa está en evidencia/02_desnormalizacion_top_categorias.txt.

b) Patrón elegido: columna/tabla precalculada con disparadores

Se eligió una tabla resumen **resumen_venta_diaria_categoria** (una fila por día y categoría) mantenida por disparadores. La evidencia de (a) muestra que casi todo el costo es localizar los pedidos del día recorriendo 600.000 filas y luego resolver tres joins y una agregación sobre miles de detalles, trabajo que se repite completo en cada una de las muchas ejecuciones por minuto aunque entre ellas cambien pocos pedidos; un índice sobre pedido(fecha), medido como alternativa, solo baja la mediana a 8,556 ms porque sigue uniendo y agregando 4.091 detalles. La vista materializada se descartó porque el panel exige tiempo real y cada REFRESH recalcula 1.500.000 detalles. La sincronización la garantizan disparadores AFTER ... FOR EACH STATEMENT sobre detalle_pedido (alta, modificación, borrado y TRUNCATE), pedido (fecha y baja lógica) y producto (categoría), que aplican un delta agregado con INSERT ... ON CONFLICT DO UPDATE en la misma transacción que el cambio, más el script de auditoría de (e). Es reversible sin pérdida: las tablas fuente no se

modifican, el resumen se reconstruye con fn_tp6_reconstruir_resumen() y eliminarlo (DROP TABLE ... CASCADE y funciones fn_tp6_*) no borra ningún dato.

c) Implementación

Tabla (dia, id_categoria, total_vendido, lineas, actualizado_en), PK (dia, id_categoria) y FK a categoria; fn_tp6_dia_negocio() fija el día en Buenos Aires; carga inicial de 4.380 filas bajo LOCK TABLE; seis disparadores por sentencia con tablas de transición, que agrupan los deltas por (día, categoría) y los aplican ordenados. Script completo en tp_desnormalizacion_top_categorias.sql, sección 2.

d) Consulta sobre la estructura desnormalizada

```
SELECT c.nombre AS categoria, r.total_vendido
FROM resumen_venta_diaria_categoria r
JOIN categoria c ON c.id_categoria = r.id_categoria
WHERE r.dia = CURRENT_DATE AND r.lineas > 0
ORDER BY r.total_vendido DESC
LIMIT 5;
```

```
== 5.2(d) EXPLAIN ANALYZE leyendo el resumen (DESPUÉS) ==
 QUERY PLAN
------------------------------------------------------------
 Limit  (actual time=0.025..0.027 rows=5 loops=1)
   Buffers: shared hit=10
   ->  Sort  (actual time=0.025..0.026 rows=5 loops=1)
         Sort Key: r.total_vendido DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=10
         ->  Hash Join  (actual time=0.015..0.021 rows=12 loops=1)
               Hash Cond: (r.id_categoria = c.id_categoria)
               Buffers: shared hit=10
               ->  Bitmap Heap Scan on resumen_venta_diaria_categoria r  (actual time=0.006..0.010 rows=12 loops=1)
                     Recheck Cond: (dia = CURRENT_DATE)
                     Filter: (lineas > 0)
                     Heap Blocks: exact=7
                     Buffers: shared hit=9
                     ->  Bitmap Index Scan on pk_resumen_venta_diaria_categoria  (actual time=0.004..0.004 rows=12 loops=1)
                           Index Cond: (dia = CURRENT_DATE)
                           Buffers: shared hit=2
               ->  Hash  (actual time=0.005..0.005 rows=12 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     Buffers: shared hit=1
                     ->  Seq Scan on categoria c  (actual time=0.001..0.003 rows=12 loops=1)
                           Buffers: shared hit=1
 Planning:
   Buffers: shared hit=4
 Planning Time: 0.074 ms
 Execution Time: 0.037 ms
(26 rows)
```

Captura 4 - EXPLAIN (ANALYZE, BUFFERS) DESPUÉS: 0,037 ms, 10 buffers.

Medida	Antes (4 tablas)	Después (resumen)
Execution Time (EXPLAIN ANALYZE)	33,129 ms	0,037 ms
Mediana de 30 ejecuciones	28,695 ms	0,017 ms
Nodo dominante	Parallel Seq Scan on pedido (600.000 filas leídas, 1.643 útiles)	Bitmap Heap Scan on resumen_venta_diaria_categoria vía Bitmap Index Scan on su PK (12 filas)
Buffers	22.645 hit + 810 read	10 hit

Ambas consultas devuelven el mismo top 5 (Almacén, Bebidas, Golosinas, Mascotas, Snacks, con los mismos montos), y los controles EXCEPT en ambos sentidos, sin LIMIT, para el día y para los últimos 7 días dan 0:

```
== Equivalencia: original vs. resumen, hoy y últimos 7 días, sin LIMIT (esperado 0 en los cuatro) ==
            control             | filas
--------------------------------+-------
 hoy: original EXCEPT resumen    |     0
 hoy: resumen EXCEPT original    |     0
 7 días: original EXCEPT resumen |     0
 7 días: resumen EXCEPT original |     0
(4 rows)
```

Captura 5 - Equivalencia de resultados entre la consulta original y la desnormalizada.

Costo aceptado: insertar 1.000 detalles pasa de 8,485 ms a 10,889 ms de mediana (+2,4 ms, ≈ 28 %), y las ventas del mismo día y categoría se serializan sobre la misma fila del resumen hasta el COMMIT. Se justifica porque cada lectura del panel, que ocurre muchas veces por minuto, ahorra ≈ 28 ms.

```
== Original (4 tablas), 30 ejecuciones ==
 minimo_ms | mediana_ms | maximo_ms
-----------+------------+-----------
    28.197 |     28.695 |    32.629
(1 row)

== Desnormalizada (resumen), 30 ejecuciones ==
 minimo_ms | mediana_ms | maximo_ms
-----------+------------+-----------
     0.016 |      0.017 |     0.024
(1 row)

           escenario            | minimo_ms | mediana_ms | maximo_ms
--------------------------------+-----------+------------+-----------
 sin disparador (deshabilitado) |     8.336 |      8.485 |     8.950
 con disparador de resumen      |    10.652 |     10.889 |    15.365
(2 rows)
```

Captura 6 - Mediciones repetidas y costo de escritura (evidencia/03_mediciones_repetidas.txt).

e) Auditoría de sincronización

Recalcula desde detalle_pedido, pedido y producto el total y las líneas por día y categoría y los compara con FULL JOIN contra el resumen. Sobre la base migrada devuelve 0 filas. Las pruebas de sincronización (revertidas) auditan después de cada operación; el control negativo corrompe el resumen a mano y la auditoría lo detecta.

```
== 5.2(e) Auditoría completa (esperado: 0 filas) ==
 dia | id_categoria | total_fuente | total_resumen | lineas_fuente | lineas_resumen
-----+--------------+--------------+---------------+---------------+----------------
(0 rows)
```

Captura 7 - Auditoría sobre la base migrada: resultado vacío.

Prueba de sincronización (dentro de BEGIN ... ROLLBACK)	Diferencias de auditoría
alta	0
cantidad	0
baja logica detalle	0
fecha ayer	0
fecha hoy	0
baja logica 100 pedidos	0
recategorizacion	0
borrado fisico	0
corrupcion manual	1

Diferencias detectadas después de cada prueba, transcriptas de evidencia/02_desnormalizacion_top_categorias.txt (0 en todas; 1 en el control negativo, que corrompe el resumen a mano).

Declaración de uso de IA

Se usó Claude (Anthropic) como asistente para redactar los scripts, el generador de datos y el informe. Todo se verificó ejecutándolo en PostgreSQL 16.15 y las cifras provienen de las salidas en tp6/evidencia/.